CareerAI

Your Agentic AI Career Mentor

7 Autonomous AI Agents · Personalized Placement Coaching · Proactive Interventions

Team Virtus

Team Members

Lovejeet Singh · Sachit Babbar · Mohammad Asim · Manveer Singh

Hackathon: GeekVerse — GeeksforGeeks X LPU

Category: Open Idea

Date: April 2026

Table of Contents

1. Problem Statement
2. Our Approach & Solution
3. System Architecture
4. Tech Stack
5. Why Not LangChain?
6. The 7 AI Agents — Deep Dive
7. Live Jobs (Apify LinkedIn Scraper)
8. Database Schema (Supabase PostgreSQL)
9. Gamification System
10. Interactive Roadmap Maker
11. Security & Auth
12. Frontend Architecture
13. Backend Architecture
14. End-to-End User Workflows
15. API Routes Reference
16. Future Scope

1. Problem Statement

Engineering students in India face a **fragmented, overwhelming, and generic** placement preparation experience:

- **No personalized guidance** — Students use the same YouTube playlists and roadmaps regardless of their current skill level, goals, or target companies.
- **No self-awareness** — Students don't know their actual strengths, weaknesses, or placement readiness.

- **Disconnected tools** — Resume analysis, mock interviews, task planning, and job search happen on separate platforms with no integration.
- **No real-time job data** — Students prepare blindly without knowing what companies are actually hiring right now.
- **Inconsistent effort** — Without accountability (streaks, tracking, gamification), motivation drops within weeks.

> **The Core Gap**
>
> There is no single platform that combines AI-powered profile analysis, personalized task planning, real-time job matching, mock interviews with feedback, and career mentorship — all adapting to the student's progress in real time.

2. Our Approach & Solution

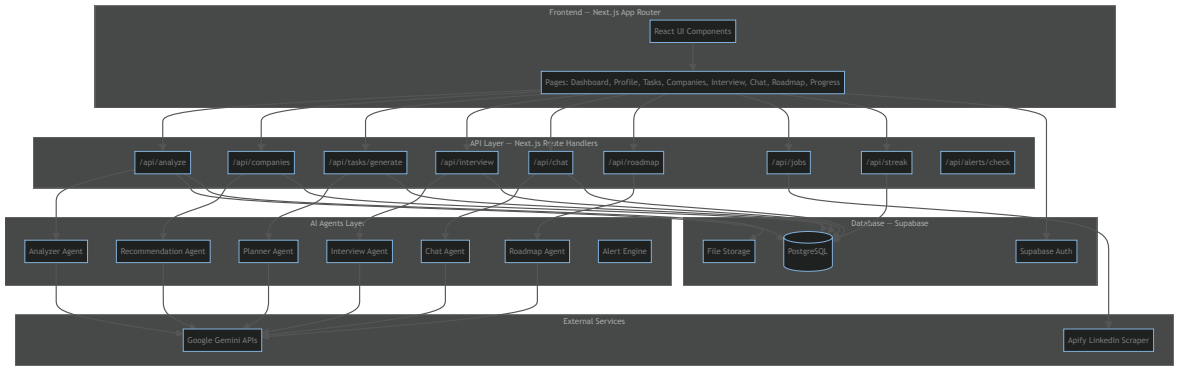
CareerAI is a **multi-agent AI platform** where 7 specialized AI agents collaborate to provide an end-to-end career coaching experience. Each agent has a focused responsibility, a typed input/output schema, and communicates through a shared database.

Key Design Principles

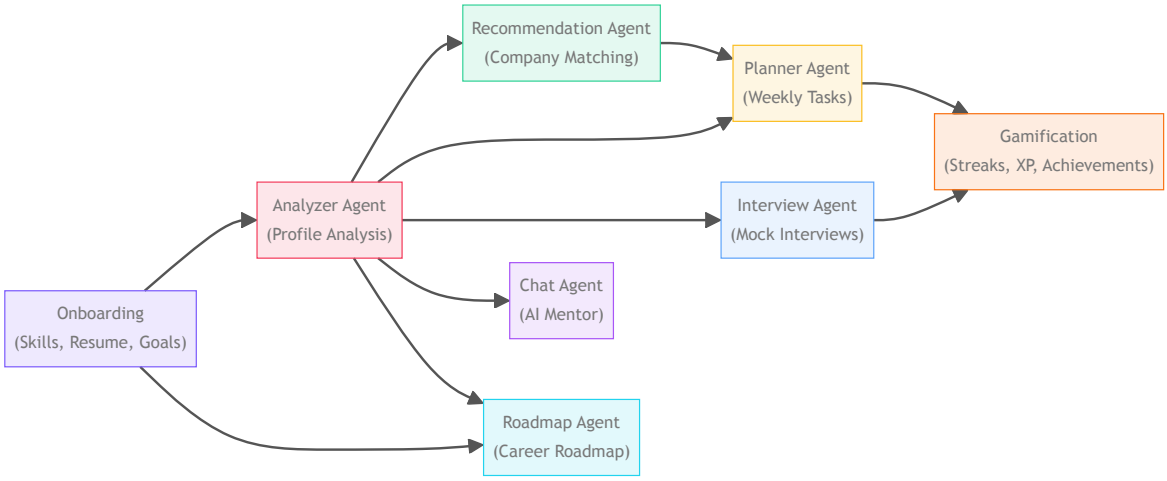
1. **Agent Specialization** — Each AI agent has one job and does it expertly, rather than a single monolithic prompt doing everything.
2. **Real Data Pipeline** — Profile → Analysis → Recommendations → Tasks → Interviews. Each agent's output feeds into the next.
3. **Hybrid Intelligence** — AI-generated company matches + real-time LinkedIn job data from Apify.
4. **Honest AI** — We instruct Gemini to be *brutally honest* about readiness, not sugarcoat assessments.
5. **Gamification** — Streak tracking, XP, levels, achievements — all computed from real database activity (zero mock data).

3. System Architecture

High-Level Architecture



Agent Data Flow (Pipeline)



4. Tech Stack

CATEGORY	TECHNOLOGY	WHY WE CHOSE IT
Framework	Next.js 16 (App Router)	Server-side rendering, API routes in the same repo, React Server Components
Data Fetching	SWR	Global caching for instant tab-switching and background data revalidation

CATEGORY	TECHNOLOGY	WHY WE CHOSE IT
State Persistence	**react-use**	Uses `useLocalStorage` to persist complex AI roadmaps across sessions securely
AI Model	**Google Gemini Multi-Tier**	Dynamic fallback chain: 3.1-flash → 2.5-flash-lite → 2.5-flash for max uptime
AI SDK	**@google/genai**	Official Google SDK — lightweight, direct API calling (no LangChain overhead)
Database	**Supabase (PostgreSQL)**	Auth + DB + Storage in one service, Row-Level Security, real-time subscriptions
Auth	**Supabase Auth**	Google OAuth, magic links, session management — zero custom auth code
Job Data	**Apify** (LinkedIn Scraper)	Real-time LinkedIn job scraping, no cookies needed, reliable API
Styling	**Tailwind CSS v4**	Utility-first CSS, dark mode, rapid prototyping
UI Library	**shadcn/ui + Radix**	Accessible, composable, styled with Tailwind
Animations	**Framer Motion**	Declarative animations, AnimatePresence for page transitions
Charts	**Recharts**	React-native charting for progress visualizations
PDF Parsing	**pdf-parse**	Extracts text from uploaded resume PDFs for AI analysis
Language	**TypeScript**	Type-safe agent I/O schemas, catches errors at compile time

5. Why Not LangChain?

This is a common question from judges. Here's our **deliberate technical reasoning**:

LangChain Problems

- **Massive bundle** — adds 2-5MB+ to server, significant cold start penalty on serverless (Vercel)
- **Abstraction overhead** — chains, agents, tools, memory managers — we don't need 90% of it
- **Opaque debugging** — when a chain fails, you're debugging 5 layers of abstraction
- **Version instability** — frequent breaking API changes
- **Vendor lock-in** — LangChain's abstractions make it harder to switch models

Our Approach Instead

- **Direct API calls** — `@google/genai` SDK, ~50KB, zero cold start
- **Single shared function** — `callGemini()` — all 6 agents call through one 40-line function
- **TypeScript-typed I/O** — each agent has a defined `Input` / `Output` type
- **JSON mode** — Gemini's native `responseMimeType: 'application/json'`, no parsing hacks
- **Agent = function** — each agent is just an exported `async function` with a system prompt

The Core: `callGemini()` — Our "Framework"

Instead of LangChain's 100+ classes, we have one 40-line function that every agent calls through:

```
// src/lib/agents/gemini.ts — The ENTIRE "framework"
export async function callGemini<T>({
  systemPrompt,     // Agent's role & rules
  userPrompt,       // Context + question
  jsonMode = true,  // Gemini native JSON output
  temperature = 0.7,
  maxTokens = 8192,
}: GeminiCallOptions): Promise<T> {
  const response = await ai.models.generateContent({
    model: 'gemini-2.5-flash',
    contents: userPrompt,
    config: {
      temperature,
      maxOutputTokens: maxTokens,
```

```
      responseMimeType: jsonMode ? 'application/json' : undefined,
      systemInstruction: systemPrompt,
    },
  });
  return jsonMode ? JSON.parse(response.text) : response.text;
}
```

We also engineered a highly resilient custom fallback chain directly into the wrapper. If the primary model hits a quota limit or fails, it automatically falls back to `gemini-2.5-flash-lite`, ensuring 100% platform uptime during high-traffic hackathon judging, without the need for bulky third-party retry plugins.

> **Result**
>
> 7 specialized agents, zero external framework dependencies. Total agent layer code: ~500 lines. A LangChain equivalent would be 2000+ lines with 15+ imported classes.

6. The 7 AI Agents — Deep Dive

Each agent follows the same pattern: **System Prompt + Typed Input → Gemini API → Typed Output**.



🧠 Agent 1: Profile Analyzer

`src/lib/agents/analyzer.ts → /api/analyze`

Purpose: Analyzes a student's complete profile (skills, resume, projects, education) and produces a placement readiness assessment.

Temperature: 0.4 (low — we want consistent, factual analysis, not creative output)

INPUT

- Skills list + self-ratings (1-10)
- Projects (name, tech stack, description)
- Education level & field
- Experience level (fresher/intern/1-3yr)
- Career goals
- Resume text (extracted from PDF)

OUTPUT

- `extracted_skills[]` — name, level, confidence
- `strengths[]` — skill + evidence
- `weaknesses[]` — skill + reason
- `missing_skills[]` — skill + importance
- `readiness_score` — 0-100
- `summary` — 2-3 sentence assessment

Storage: Output is saved to the `analyses` table in Supabase for historical tracking.

🏢 Agent 2: Company Recommender

`src/lib/agents/recommendation.ts → /api/companies`

Purpose: Matches the student with REAL Indian tech companies based on their analysis results. Split into "Current Fit" (hire-ready now) and "Target" (aspirational).

Temperature: 0.5 (balanced — needs to know real companies but also personalize)

INPUT (FROM ANALYZER OUTPUT)

- Readiness score
- Extracted skills + strengths + weaknesses
- Missing skills + importance
- Career goals + experience level
- Preferred locations

OUTPUT

- `current_fit_companies[]` — name, role, match%, skills
- `target_companies[]` — name, gap skills, timeline
- `skill_gaps[]` — current vs required level
- `roadmap_to_target[]` — month-by-month plan

Key Constraint: System prompt mandates ONLY real Indian-market companies (Google, Flipkart, TCS, Razorpay, etc.) — no fictional companies.

📋 Agent 3: Task Planner

`src/lib/agents/planner.ts → /api/tasks/generate`

Purpose: Generates a personalized weekly task plan based on the analysis + company recommendations. Tasks reference real platforms (LeetCode, GFG) and real problem names.

Temperature: 0.6 (creative enough for varied tasks, structured enough for actionable plans)

INPUT (FROM ANALYZER + RECOMMENDER)

- Analysis results (score, skills, weaknesses)
- Recommendation results (target companies, gaps)
- Current week number
- Available hours/day
- Career goals

OUTPUT

- `weekly_plan.theme` — week's focus area
- `tasks[]` — title, description, category
- Each task: difficulty, type (daily/weekly/milestone)
- Day assignment (1-7), priority, estimated minutes

- Categories: dsa, web_dev, system_design, soft_skills

Storage: Tasks saved to `tasks` table. Completion tracked with timestamps for streak calculation.

🎤 Agent 4: Interview Conductor

`src/lib/agents/interview.ts → /api/interview`

Purpose: Two sub-functions: (1) **Generate** interview questions by topic + difficulty, and (2) **Evaluate** student answers with scoring and feedback.

Temperatures: Generation = 0.6 (creative questions), Evaluation = 0.3 (consistent scoring)

GENERATE INPUT

- Topic (DSA, System Design, etc.)
- Difficulty (easy/medium/hard)
- Student level
- Number of questions

EVALUATE OUTPUT

- `score` — 0-10 per question
- `feedback` — detailed review
- `topics_covered[]` + `topics_missed[]`
- `improvement_tip` — actionable advice

Storage: Questions, answers, and evaluations stored in `interviews` table. Overall score computed and saved.

💬 Agent 5: AI Mentor (Chat)

`src/lib/agents/chat.ts → /api/chat`

Purpose: Conversational AI mentor with personality. Knows the student's skills, goals, and readiness score. Provides career advice, doubt solving, and encouragement.

Temperature: 0.7 (conversational, friendly, but accurate)

INPUT **OUTPUT**

- Student message
- Last 10 chat messages (context window)
- Student context: skills, readiness score, goals

- `response` — markdown-formatted reply
- `suggested_actions[]` — follow-up buttons

Storage: Full chat history in `chat_messages` table. Loaded on page open, persists across sessions.



Agent 6: Roadmap Architect

`src/app/api/roadmap/route.ts → /api/roadmap`

Purpose: Generates a brutally honest, week-by-week career preparation roadmap based on the student's dream company, timeline, and current skills. Includes feasibility assessment and skill gap analysis.

Temperature: 0.6 · **Max tokens:** 16,000 (large output for detailed weekly plans)

INPUT

- Dream company + target role
- Timeline (1-12 months)
- Available hours/day
- Focus areas selected by user
- Full profile + latest AI analysis + project count

OUTPUT

- `overview` — feasibility, readiness projection
- `phases[]` — each with weeks, each week with tasks
- `keySkillsToAcquire[]` — priority-ranked gaps
- `interviewPrep` — DSA target, system design topics
- `warnings[]` — honest challenges

Key Feature: If the dream is unrealistic in the timeframe, the AI says so and suggests alternative companies.



Agent 7: Proactive Alert Engine

`src/app/api/alerts/check/route.ts → /api/alerts/check`

Purpose: A rule-based autonomous agent that runs silently on dashboard load. It evaluates the user's task history, skill progression, and inactivity, and intelligently calculates if it needs to trigger an intervention.

INPUT

- Missed deadlines / Overdue Tasks
- Unpracticed Weaknesses
- Days since last consecutive activity

OUTPUT

- `alerts[]` — Dynamic priority queue (critical > warning > action)

Key Feature: Behaves proactively rather than reactively. It does NOT wait for a user prompt; it intercepts the session and demands action if the user begins slacking off.

7. Live Jobs — Apify LinkedIn Scraper

Why Apify?

Our AI agents generate *predicted* company matches, but students also need **actual, live job listings** to apply to. We integrated Apify's LinkedIn scraper to provide real-time job data.

Actor Used

PROPERTY	VALUE
Actor ID	`curious_coder/linkedin-jobs-scraper`
Total Users	44,000+
Rating	4.9 (67 reviews)
Pricing	$1.00 / 1,000 results (pay-per-use)
Cookies Required	**No** — scrapes public LinkedIn data
LinkedIn Account Risk	**Zero** — no login needed

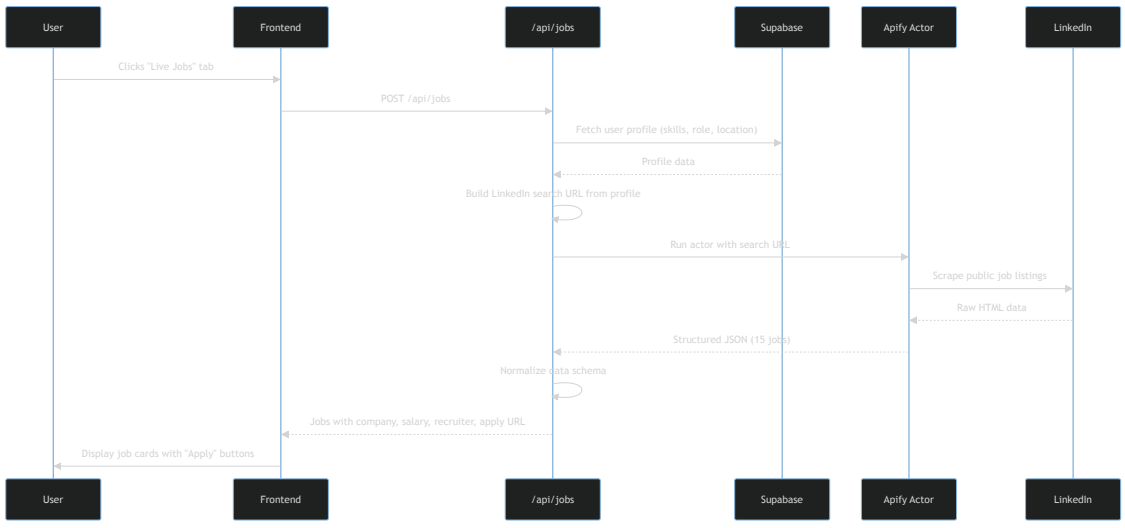
Why This Actor Over Others?

We evaluated multiple LinkedIn scrapers on Apify:

ACTOR	COST	COOKIES?	USERS	RATING	DECISION
`curious_coder/linkedin-jobs-search-scraper`	$30/month	Yes	3.6K	3.7	❌ Expensive + risky
`bebity/linkedin-jobs-scraper`	Varies	No	28K	4.2	❌ Lower accuracy

ACTOR	COST	COOKIES?	USERS	RATING	DECISION
curious_coder/linkedin-jobs-scraper	$1/1K	No ✅	44K	4.9	✅ Best value + quality

How It Works



Auto-Generated Search URL

The API route automatically builds a LinkedIn search URL from the user's profile:

```
// Inputs from Supabase profile
const keywords = profile.preferred_role || skills.slice(0,3).join(' ');
const location = profile.preferred_locations?.[0] || 'India';
const experience = getExperienceFilter(profile.experience_level);

// Generated URL
https://www.linkedin.com/jobs/search/?keywords=Software+Engineer&location=Bangalore&f_TPR=r6048
```

Data Returned Per Job

Each job card displays: title, company name + logo, location, salary range, employment type (full-time/contract), seniority level, post date, description preview, applicant count, recruiter name + photo + LinkedIn profile, and a direct **Apply** link.

8. Database Schema (Supabase PostgreSQL)

8 tables with **Row-Level Security** on every table. The schema is designed for agent-driven data flow — each agent reads/writes to specific tables, enabling autonomous operation.

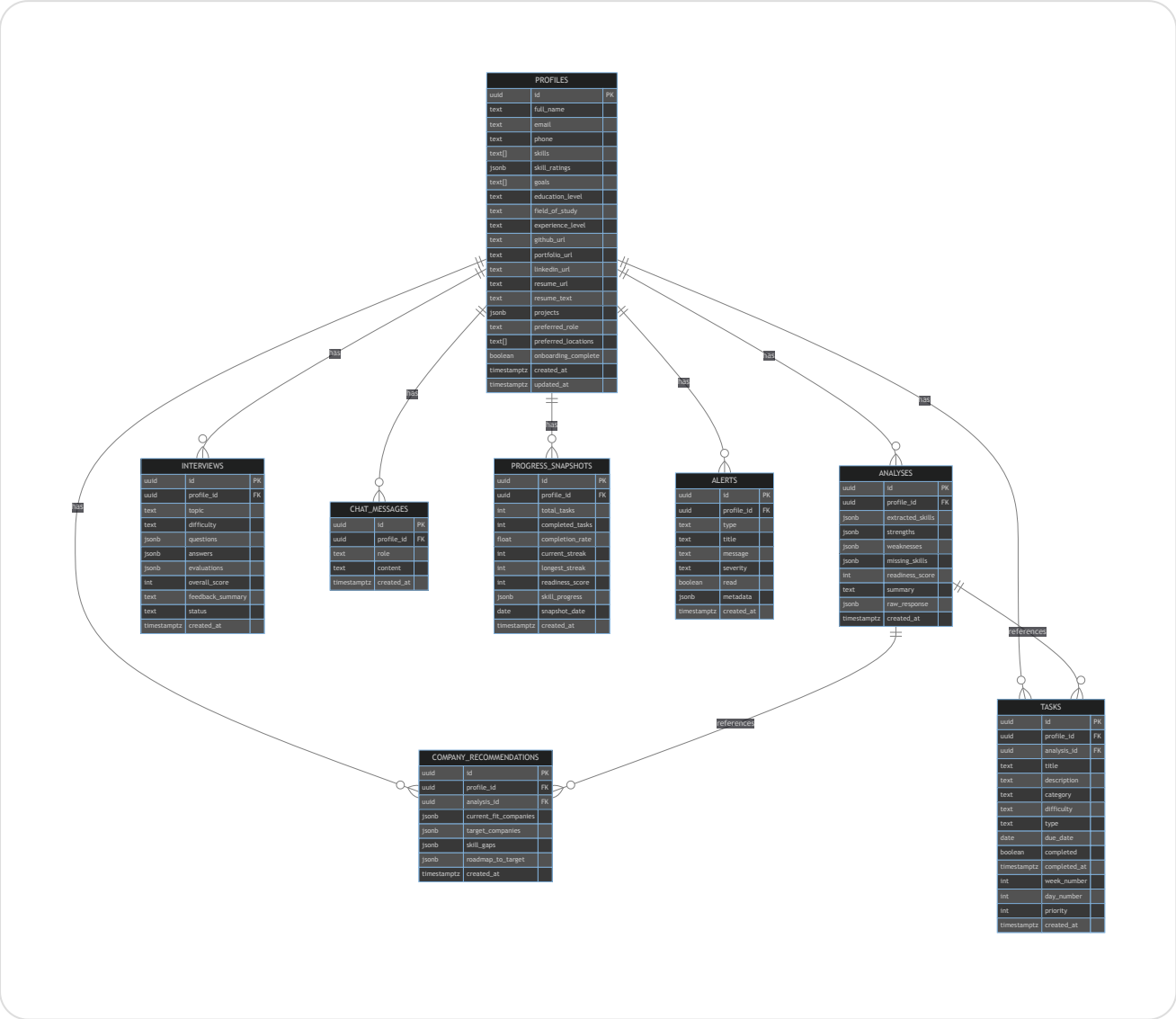


Table-by-Table Breakdown

TABLE	PURPOSE	WRITTEN BY	KEY COLUMNS
`profiles`	Core user data — extends `auth.users`	Onboarding flow + user edits	skills[], skill_ratings, projects (JSONB), resume_text, preferred_role, preferred_locations[]

TABLE	PURPOSE	WRITTEN BY	KEY COLUMNS
`analyses`	AI-generated readiness assessments	Profile Analyzer Agent	extracted_skills, strengths, weaknesses, missing_skills, readiness_score (0-100)
`company_recommendations`	AI company matching results	Recommender Agent	current_fit_companies, target_companies, skill_gaps, roadmap_to_target
`tasks`	AI-generated daily/weekly practice tasks	Task Planner Agent	title, category (dsa/web_dev/etc.), difficulty, type, due_date, completed, week_number, day_number
`interviews`	Mock interview sessions with evaluations	Interview Mentor Agent	topic, questions (JSONB), answers (JSONB), evaluations (JSONB), overall_score, feedback_summary
`chat_messages`	Full AI mentor conversation history	Chat Mentor Agent	role (user/assistant), content
`alerts`	Proactive intervention notifications	Alert Engine (autonomous)	type, title, message, severity, read (bool), metadata (JSONB)
`progress_snapshots`	Point-in-time progress records	Streak API route	total_tasks, completed_tasks, completion_rate, current_streak,

TABLE	PURPOSE	WRITTEN BY	KEY COLUMNS
			longest_streak, skill_progress

Security Architecture

Row-Level Security (RLS)

All 8 tables have RLS enabled. Every policy enforces `auth.uid() = profile_id` (or `auth.uid() = id` for profiles). Users can only read and write their own data — no cross-user data leaks, even with direct Supabase API access.

Auto-Provisioning Trigger

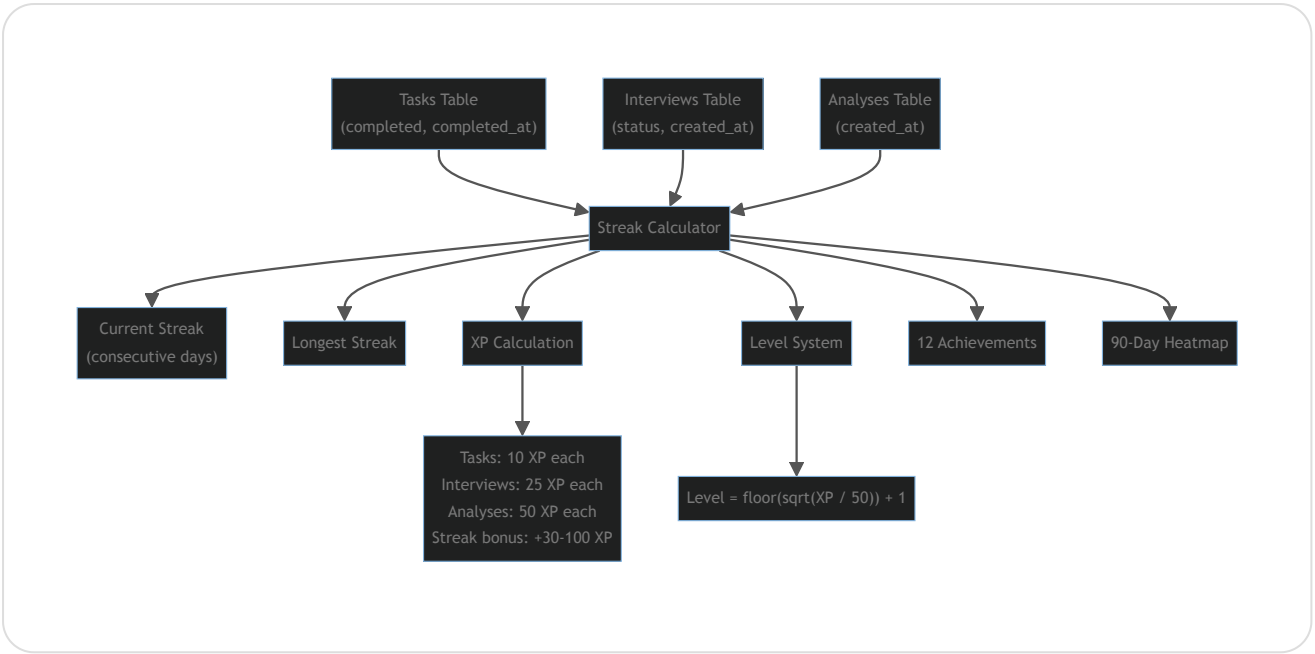
A PostgreSQL trigger (`handle_new_user()`) fires on `auth.users INSERT` to auto-create a profile row, extracting `full_name` from Google OAuth metadata. Zero manual profile creation needed.

Storage Bucket

A dedicated `resumes` bucket stores uploaded PDF files. Only authenticated users can upload; files are publicly readable for AI processing.

9. Gamification System

All gamification data is computed in real-time from the database — zero hardcoded or mock values.



METRIC	SOURCE	CALCULATION
Current Streak	`tasks.completed_at` , `interviews.created_at`	Count consecutive days with activity (today/yesterday as anchor)
XP	All 3 tables	Tasks × 10 + Interviews × 25 + Analyses × 50 + streak bonus
Level	Total XP	`floor(sqrt(totalXP / 50)) + 1` (exponential curve)
Heatmap	Activity dates	Last 90 days, count per day
Achievements	Milestone counts	12 achievements (First Step, Task Master, Grinder, On Fire, Week Warrior, etc.)

Visual Feedback Loop

To deepen the gamification loop, we introduced a globally injected `<CelebrationTracker />` component. It silently observes SWR state changes across the entire app. If a user hits a level-up or triggers a major streak (e.g., 3-day or 7-day), it intercepts the UI and fires a screen-wide confetti explosion alongside a motivational toast, greatly enhancing long-term habit building.

10. Interactive Roadmap Maker

User Input Form (Persisted)

- **Dream Company** — free text + 16 quick-select chips (Google, Amazon, Flipkart, TCS, etc.)
- **Target Role** — Software Engineer, Frontend Dev, Data Analyst, etc.
- **Timeline** — slider from 1 to 12 months
- **Hours/Day** — slider from 1h (casual) to 10h (intense)
- **Focus Areas** — multi-select: DSA, Web Dev, System Design, Projects, Soft Skills

**Note: User preferences and the generated roadmap are persisted seamlessly across sessions via `useLocalStorage`, preventing users from losing their AI-generated 12-month plans if they close the browser.*

AI Input Context

The Gemini prompt includes the student's **full profile + latest AI analysis + completed task count** to ensure the roadmap accounts for *what they already know*.

Generated Roadmap Structure



11. Security & Authentication

Authentication

- **Provider:** Supabase Auth (Google OAuth)

Data Security

- **RLS:** Row-Level Security on ALL 8 tables

- **Session:** HTTP-only cookies via `@supabase/ssr`
- **Middleware:** Next.js middleware protects all `/dashboard`, `/profile`, `/roadmap`, etc. routes
- **API Auth:** Every API route calls `supabase.auth.getUser()` and returns 401 if missing

- **Policy:** `auth.uid() = profile_id` — users see only their own data
- **API Keys:** `GEMINI_API_KEY` and `APIFY_API_TOKEN` are server-side only (never exposed to client)
- **Resume Storage:** Uploaded to Supabase Storage bucket with auth policies

12. Frontend Architecture

The frontend is built with **Next.js 16 App Router** using React Server Components and client-side interactivity where needed. Every page lives under the `(app)/` route group which shares a common authenticated layout with sidebar navigation.

Page Structure

PAGE	ROUTE	DESCRIPTION	KEY FEATURES
Landing	`/`	Marketing landing page	Animated hero, feature showcase, CTA to signup
Auth	`/auth/*`	Login / Signup	Google OAuth via Supabase, magic link support
Onboarding	`/onboarding`	5-step profile setup wizard	StepBasicInfo → StepSkills → StepGoals → StepResume → StepProjects
Dashboard	`/dashboard`	Main hub — readiness score, tasks, alerts	SWR-cached aggregated data, proactive alert engine triggers here
Profile	`/profile`	CV Analysis Report	Readiness score gauge, skill breakdown, strengths/weaknesses cards
Companies	`/companies`	AI recommendations + live LinkedIn jobs	Current Fit vs Target tabs, Apify live job cards with Apply links
Tasks	`/tasks`	AI-generated daily task management	Category filters, priority sorting, completion tracking, XP rewards
Interview	`/interview`	AI mock interview simulator	Topic/difficulty selector, real-time scoring per question,

PAGE	ROUTE	DESCRIPTION	KEY FEATURES
			feedback
Chat	`/chat`	AI Mentor conversation	Markdown rendering, context injection, suggested actions buttons
Roadmap	`/roadmap`	Personalized career roadmap generator	Dream company selector, timeline slider, phased week-by-week plan
Progress	`/progress`	Gamification dashboard	Streaks, XP, levels, 90-day heatmap, 12 achievements, confetti

UI Component Library

18 components from **shadcn/ui** (built on Radix primitives), styled with **Tailwind CSS v4**. All components are accessible (ARIA-compliant) and theme-aware (dark mode). Page transitions and micro-interactions use **Framer Motion** with `AnimatePresence`.

Data Fetching Strategy

SWR (Stale-While-Revalidate)

All data fetching uses **SWR hooks** with global cache. Switching between tabs is **instant** (shows cached data) while revalidating in the background. No loading spinners on repeat visits.

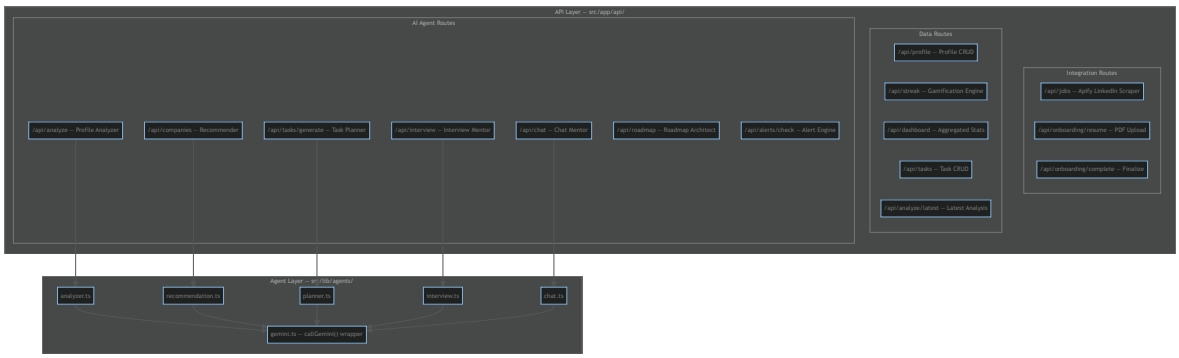
useLocalStorage Persistence

Complex AI-generated roadmaps are persisted in `localStorage` via `react-use`. Students don't lose their 12-month roadmap if they close the browser.

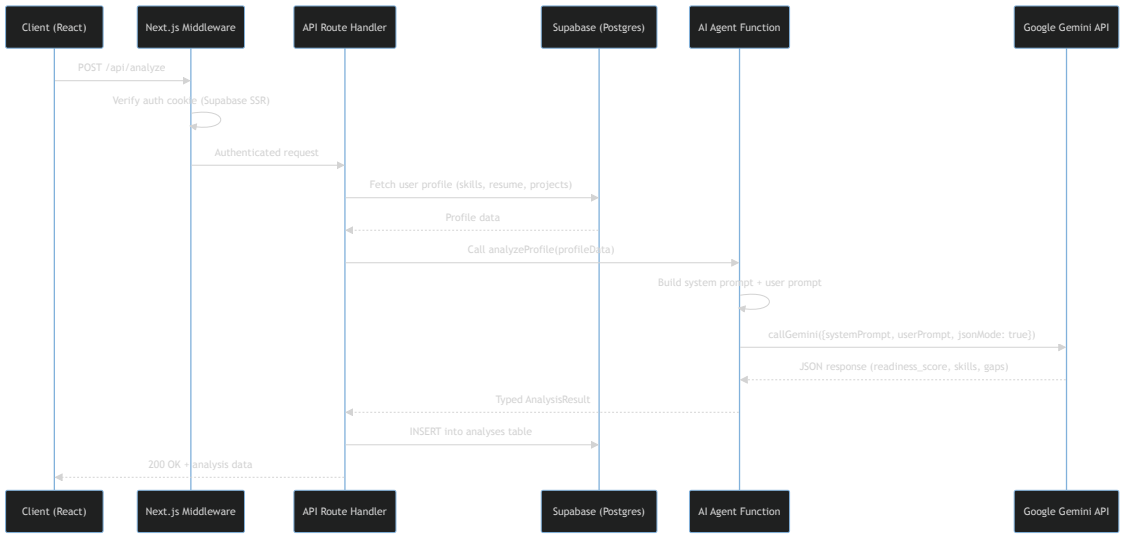
13. Backend Architecture

The backend is a **serverless API layer** built entirely with **Next.js Route Handlers** — no separate Express/FastAPI server. Every API route runs as an edge/serverless function on Vercel, auto-scaling to zero when idle.

API Route Organization



How a Typical API Request Works



Backend Design Patterns

Auth-First

Agent = Function

Every API route starts with `supabase.auth.getUser()`. If no valid session → instant `401 Unauthorized`. No unauthenticated AI calls ever reach Gemini.

Each agent is a single exported `async function` with a system prompt. No classes, no chains, no framework overhead. Input → Gemini → typed output.

Multi-Tier Fallback

The `callGemini()` wrapper chains 3 models: `gemini-3.1-flash-lite → 2.5-flash-lite → 2.5-flash`. On 429/500, it auto-retries with the next model. 100% uptime on free tier.

JSON Mode

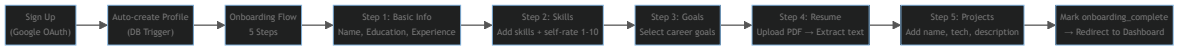
All agent calls use Gemini's native `responseMimeType: 'application/json'`. No regex parsing, no string extraction hacks — guaranteed valid JSON output.

18 API Routes Total

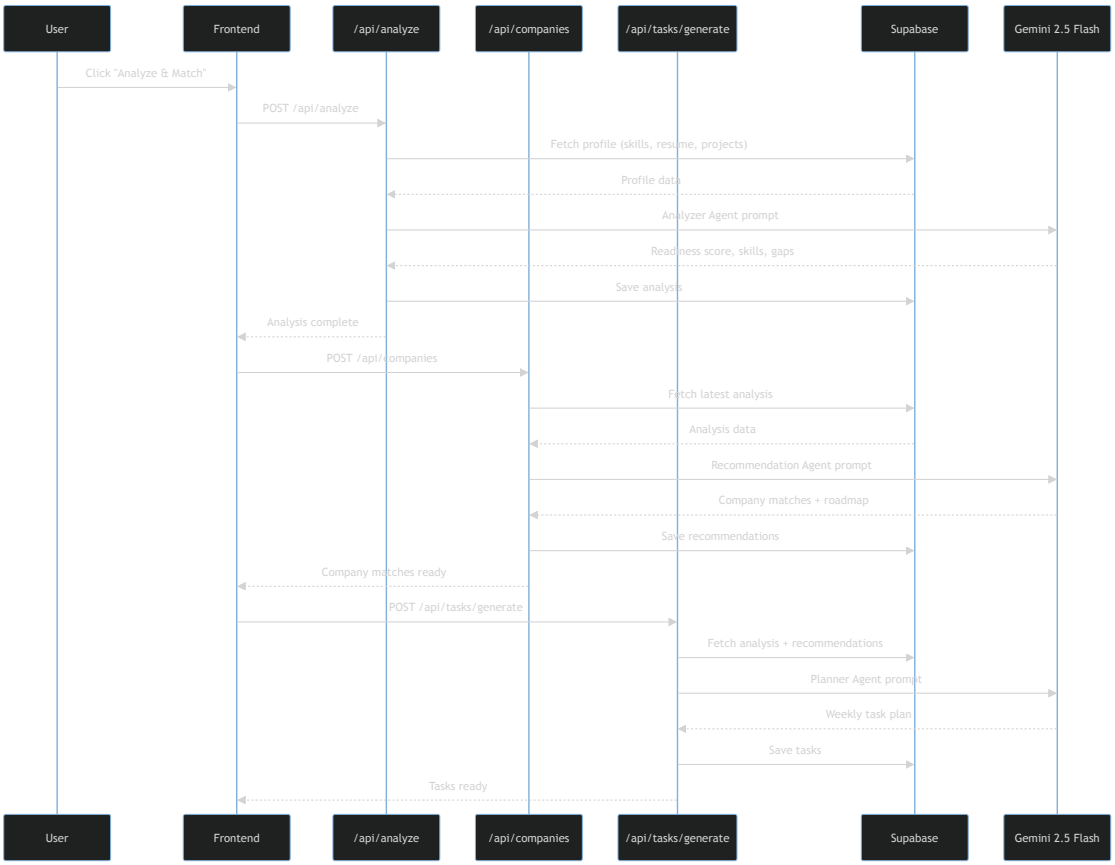
CATEGORY	COUNT	ROUTES
AI Agent Routes	7	analyze, companies, tasks/generate, interview, interview/[id]/answer, chat, roadmap, alerts/check
Data CRUD Routes	6	profile (GET/PUT), tasks (GET/PATCH/DELETE), analyze/latest, companies (GET), dashboard, streak
Integration Routes	3	jobs (Apify), onboarding/resume (PDF upload), onboarding/complete
Auth Routes	2	auth/callback (OAuth), auth/signout

14. End-to-End User Workflows

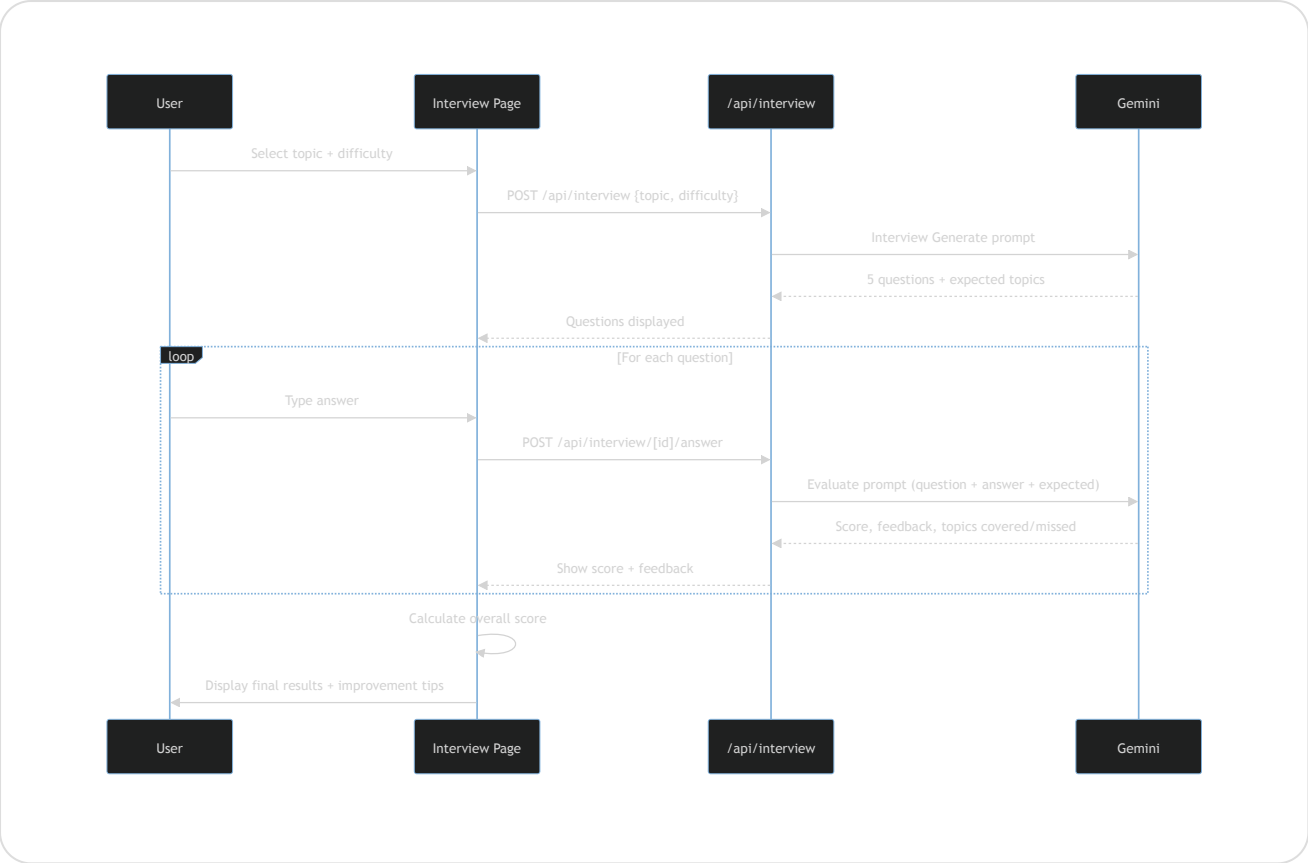
Workflow 1: New User Onboarding



Workflow 2: Full Analysis Pipeline



Workflow 3: Mock Interview



15. API Routes Reference

METHOD	ROUTE	DESCRIPTION	AUTH
GET	`/api/analyze/latest`	Fetch latest analysis results	✅
POST	`/api/analyze`	Run Analyzer Agent on profile	✅
GET	`/api/companies`	Fetch saved company recommendations	✅
POST	`/api/companies`	Run Recommendation Agent	✅
GET	`/api/tasks`	List all tasks	✅
POST	`/api/tasks/generate`	Run Planner Agent for weekly tasks	✅
PATCH	`/api/tasks/[id]`	Toggle task completion	✅

METHOD	ROUTE	DESCRIPTION	AUTH
POST	`/api/interview`	Generate interview questions	✅
POST	`/api/interview/[id]/answer`	Evaluate interview answer	✅
GET	`/api/chat`	Fetch chat history	✅
POST	`/api/chat`	Send message to AI Mentor	✅
POST	`/api/roadmap`	Generate career roadmap	✅
POST	`/api/jobs`	Scrape LinkedIn jobs via Apify	✅
GET	`/api/streak`	Gamification data (streaks, XP, achievements)	✅
GET	`/api/profile`	Fetch user profile	✅
POST	`/api/onboarding/resume`	Upload + parse resume PDF	✅
POST	`/api/onboarding/complete`	Mark onboarding as finished	✅

16. Future Scope

Analytics Dashboard

Detailed skill progression charts over time, comparison with peer averages.

Voice Interviews

Speech-to-text for mock interviews, evaluating communication clarity and confidence.

Peer Matching

Connect students preparing for the same company to form study groups.

Email Alerts

Daily streak reminders, new job matches, and progress milestones via email.

More Job Sources

Indeed, Naukri, and Internshala scrapers for broader Indian job coverage.

Mobile App

React Native companion app with push notifications for streaks and tasks.

CareerAI — Built by **Team Virtus**

Lovejeet Singh · Sachit Babbar · Mohammad Asim · Manveer Singh

GeekVerse — GeeksforGeeks X LPU · April 2026

7 AI Agents · 18 API Routes · 8 Database Tables · 0 LangChain Dependencies